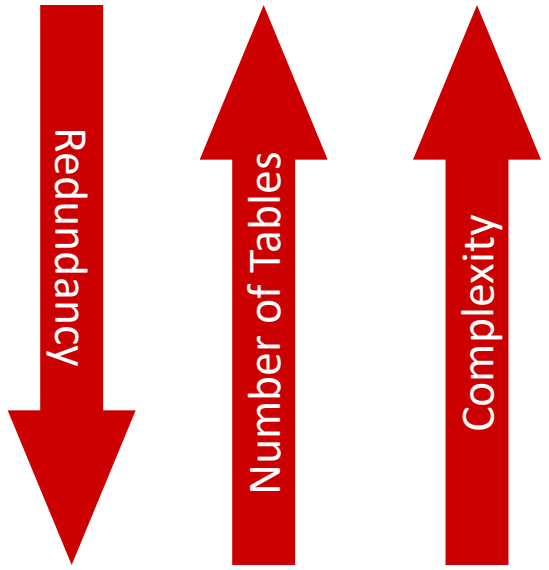
DATABASE NORMALIZATION

Definition

- This is the process which allows you to winnow out redundant data within your database.
- This involves restructuring the tables to successively meeting higher forms of Normalization.
- A properly normalized database should have the following characteristics
  - Scalar values in each fields
  - Absence of redundancy.
  - Minimal use of null values.
  - Minimal loss of information.

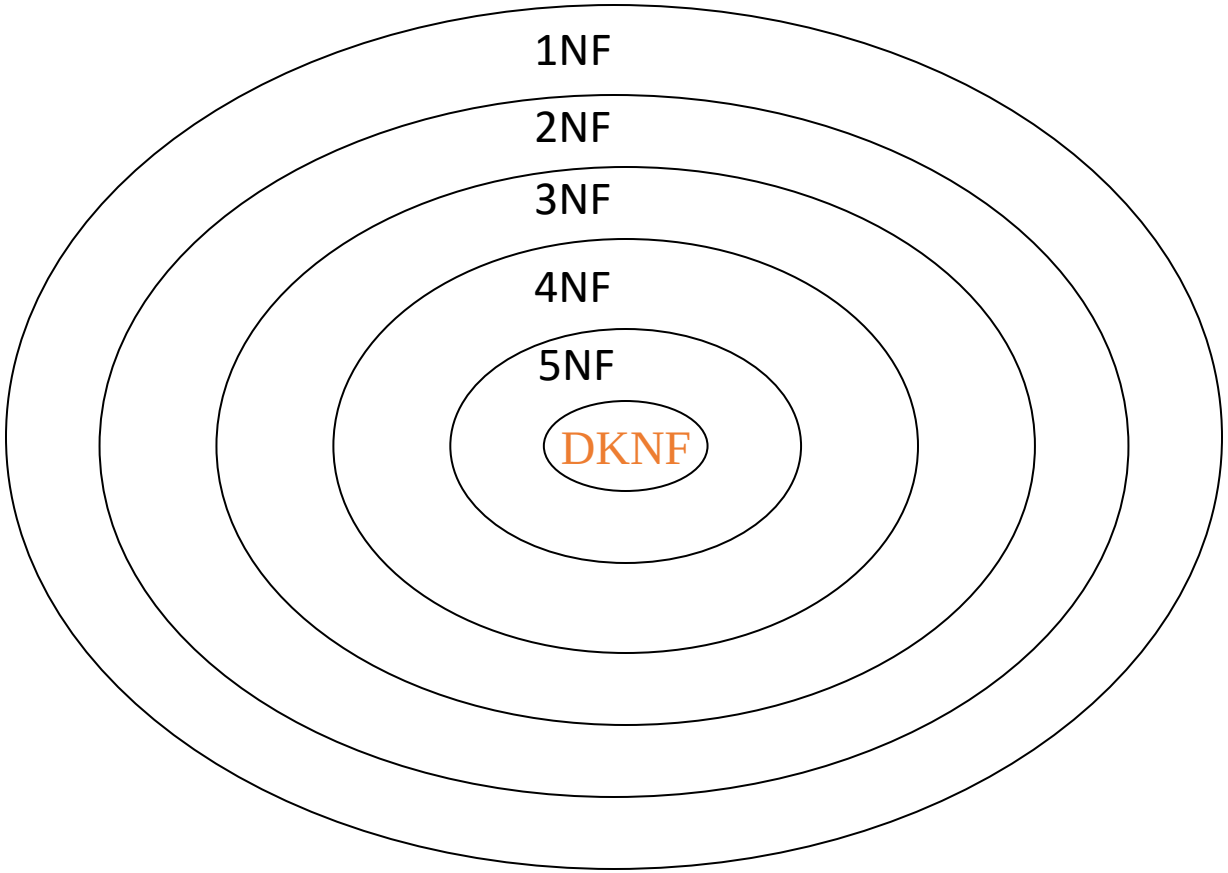
Levels of Normalization

- Levels of normalization based on the amount of redundancy in the database.
- Various levels of normalization are:
  - First Normal Form (1NF)
  - Second Normal Form (2NF)
  - Third Normal Form (3NF)
  - Boyce-Codd Normal Form (BCNF)
  - Fourth Normal Form (4NF)
  - Fifth Normal Form (5NF)
  - Domain Key Normal Form (DKNF)



Most databases should be 3NF or BCNF in order to avoid the database anomalies.

Levels of Normalization



Each higher level is a subset of the lower level

What is Normalization ?

- **NORMALIZATION** is a database design technique that organizes tables in a manner that reduces redundancy and dependency of data.
- Normalization divides larger tables into smaller tables and links them using relationships.
- The purpose of Normalization is to eliminate redundant (useless) data and ensure data is stored logically.
- The inventor of the relational model E.F.Codd proposed the theory of normalization.

Redundancy

- Row Level Redundancy:

SID	SName	Age
1	Jojo	20
2	Kit	25
1	Jojo	20

- If the SID is primary key to each row, you can use it to remove the duplicates as shown below:

SID	SName	Age
1	Jojo	20
2	Kit	25

Redundancy (Cont..)

- Column Level Redundancy:
- Now Rows are same but in column level because of Sid is primary key but columns are same.

Sid	Sname	Cid	Cname	Fid	Fname	Salary
1	AA	C1	DBMS	F1	Jojo	30000
2	BB	C2	JAVA	F2	KK	50000
3	CC	C1	DBMS	F1	Jojo	30000
4	DD	C1	DBMS	F1	Jojo	30000

What is an Anomaly?

- Problems that can occur in poorly planned, unnormalized databases where all the data is stored in one table (a flat-file database).
- Types of Anomalies:
  - Insert
  - Delete
  - Update

Anomalies in DBMS

- **Insert Anomaly :** An Insert Anomaly occurs when certain attributes cannot be inserted into the database without the presence of other attributes.
- **Delete Anomaly:** A Delete Anomaly exists when certain attributes are lost because of the deletion of other attributes.
- **Update Anomaly:** An Update Anomaly exists when one or more instances of duplicated data is updated, but not all.

Anomaly Example

- Below table University consists of seven attributes: **Sid, Sname, Cid, Cname, Fid, Fname,** and **Salary.** And the Sid acts as a key attribute or a primary key in the relation.

Table: University

Sid	Sname	Cid	Cname	Fid	Fname	Salary
1	Ram	C1	DBMS	F1	Sachin	30000
2	Shyam	C2	Java	F2	Boby	28000
3	Ankit	C1	DBMS	F1	Sachin	30000
4	saurabh	C1	DBMS	F1	Sachin	30000

Insertion Anomaly

- Suppose a new faculty joins the University, and the Database Administrator inserts the faculty data into the above table. But he is not able to insert because Sid is a primary key, and can't be NULL. So this type of anomaly is known as an insertion anomaly.

Table: University

Sid	Sname	Cid	Cname	Fid	Fname	Salary
1	Ram	C1	DBMS	F1	Sachin	30000
2	Shyam	C2	Java	F2	Boby	28000
3	Ankit	C1	DBMS	F1	Sachin	30000
4	saurabh	C1	DBMS	F1	Sachin	30000
				F3	Arun	29000

Insertion Anomaly

Delete Anomaly

- When the Database Administrator wants to delete the student details of Sid=2 from the above table, then it will delete the faculty and course information too which cannot be recovered further.

SQL:
DELETE FROM *University* WHERE *Sid=2;*

Sid	Sname	Cid	Cname	Fid	Fname	Salary
1	Ram	C1	DBMS	F1	Sachin	30000
~~2~~	~~Shyam~~	Deletion anomaly				
3	Ankit	C1	DBMS	F1	Sachin	30000
4	Saurabh	C1	DBMS	F1	Sachin	30000

Update Anomaly

- When the Database Administrator wants to change the salary of faculty F1 from 30000 to 40000 in above table University, then the database will update salary in more than one row due to data redundancy. So, this is an update anomaly in a table.

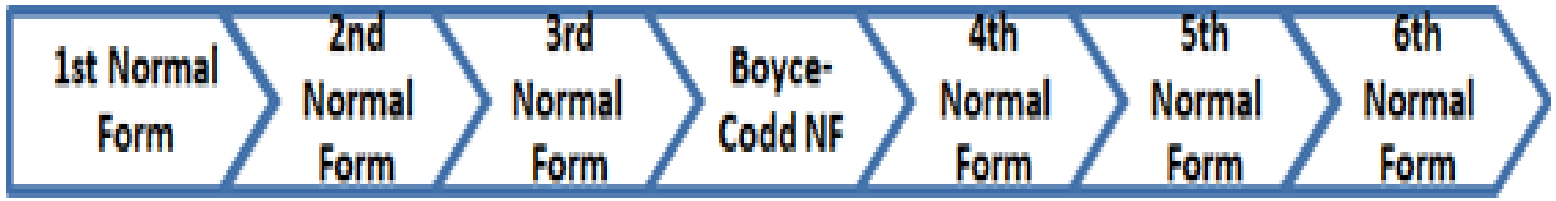
Sid	Sname	Cid	Cname	Fid	Fname	Salary
1	Ram	C1	DBMS	F1	Sachin	30000
2	Shyam	C2	Java	F2	Boby	28000
3	Ankit	C1	DBMS	F1	Sachin	30000
4	saurabh	C1	DBMS	F1	Sachin	30000

SQL:
UPDATE *University*
SET *Salary= 40000*
WHERE *Fid="F1"*;

To remove all these anomalies, we need to normalize the data in the database.

Normal forms

- The Theory of Data Normalization in SQL is still being developed further. For example, there are discussions even on 6th Normal Form. **However, in most practical applications, normalization achieves its best in 3rd Normal Form**. The evolution of Normalization theories is illustrated below-



Decomposition

- The only way to avoid the repetition-of-information problem in the i*n_dep* schema is to decompose it into two schemas – instructor and *department* schemas.
- Not all decompositions are good. Suppose we decompose

  employee(ID, name, street, city, salary)

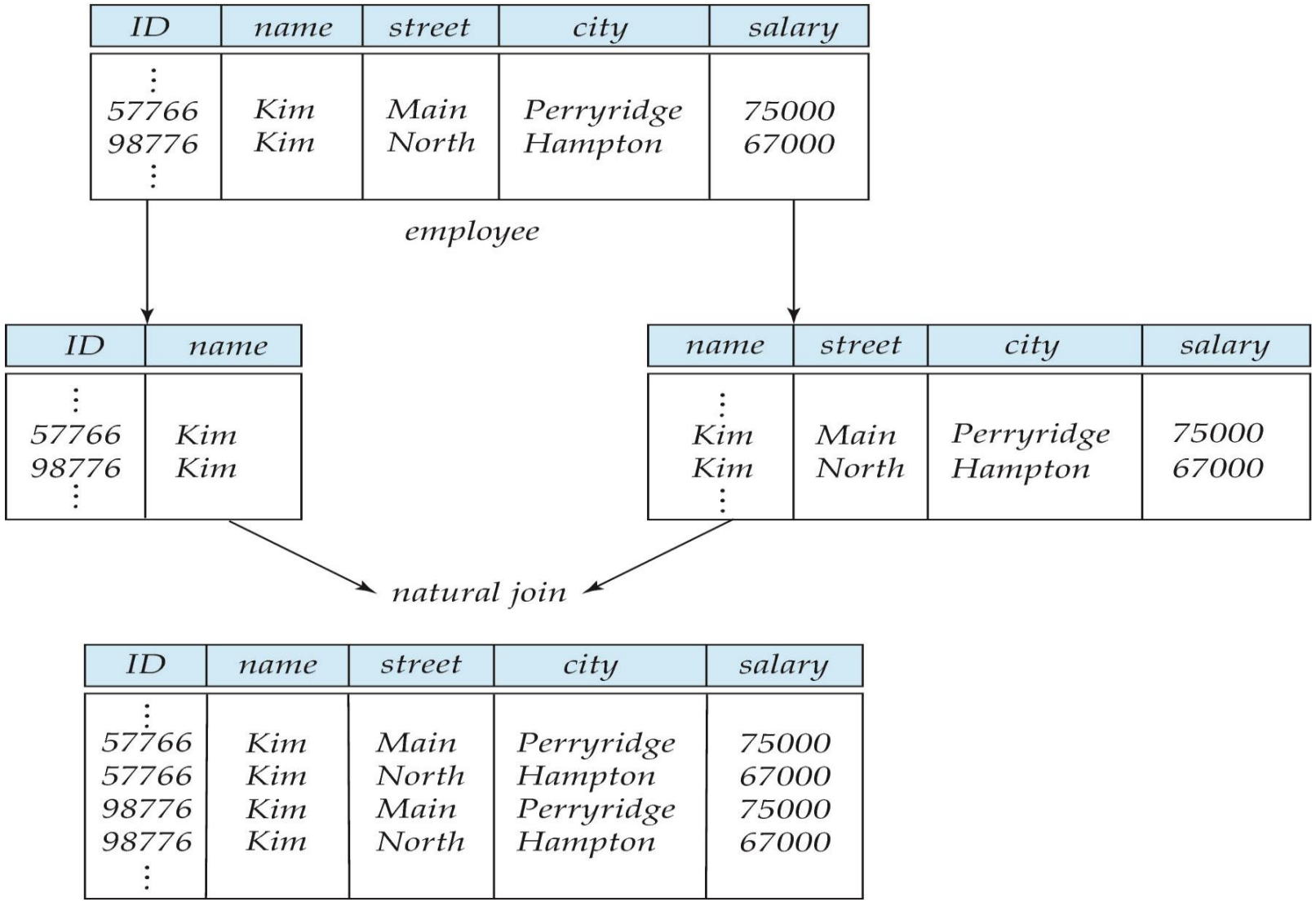
  into

  employee1 (*ID*, *name*)

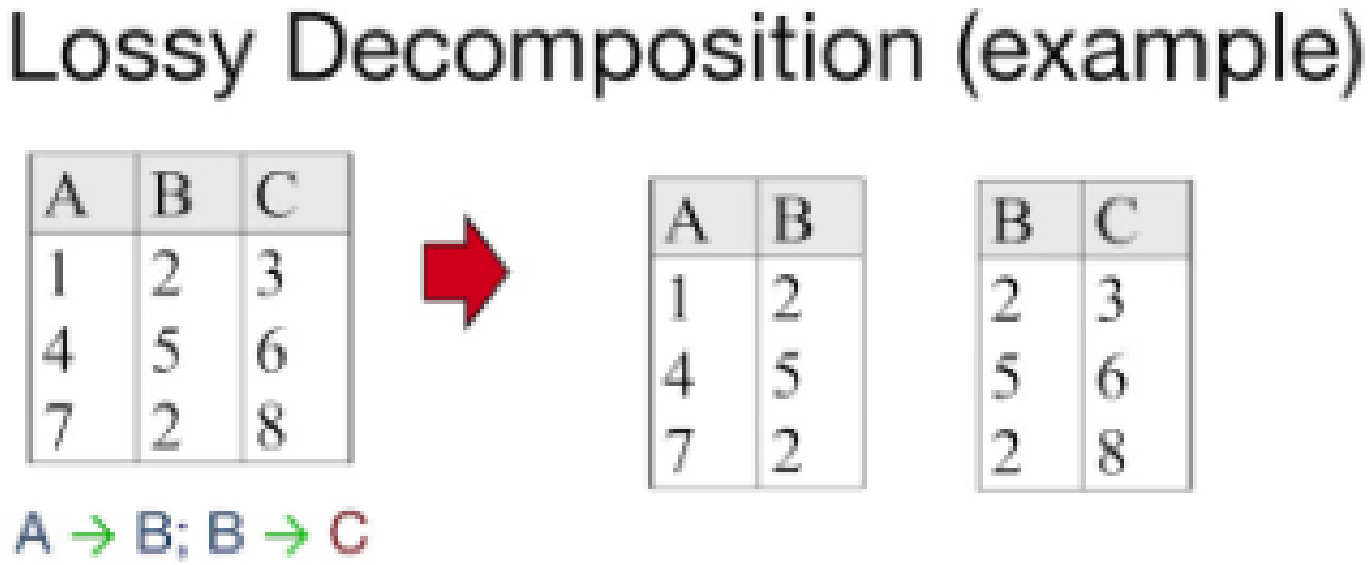
  employee2 (*name*, *street*, *city*, *salary*)

  The problem arises when we have two employees with the same name
- The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**.

A Lossy Decomposition



A Lossy Decomposition



A	B	C
1	2	3
4	5	6
7	2	8

A	B
1	2
4	5
7	2

B	C
2	3
5	6
2	8

A	B
1	2
4	5
7	2

⋈

B	C
2	3
5	6
2	8

=

A	B	C
1	2	3
4	5	6
7	2	8
1	2	8
7	2	3

Lossless Decomposition

- Let *R* be a relation schema and let R_1 and R_2 form a decomposition of R . That is R = R_1 U R_2
- We say that the decomposition is a **lossless decomposition** if there is no loss of information by replacing R with the two relation schemas R_1 U R_2
- Formally,

$$\Pi_{R_1}(r) \quad \Pi_{R_2}(r) = r$$

- And, conversely a decomposition is lossy if

$$r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

$$\bowtie$$

Example of Lossless Decomposition

- Decomposition of $R = (A, B, C)$

  $R_1 = (A, B)$ $\quad$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A (r) \bowtie \Pi_B (r)$

A	B	C
α	1	A
β	2	B

Normalization Theory

- Decide whether a particular relation *R* is in "good" form.
- In the case that a relation *R* is not in "good" form, decompose it into set of relations $\{R_1, R_2, ..., R_n\}$ such that
  - Each relation is in good form
  - The decomposition is a lossless decomposition
- Our theory is based on:
  - Functional dependencies
  - Multivalued dependencies

Functional Dependencies

- There are usually a variety of constraints (rules) on the data in the real world.
- For example, some of the constraints that are expected to hold in a university database are:
  - Students and instructors are uniquely identified by their ID.
  - Each student and instructor has only one name.
  - Each instructor and student is (primarily) associated with only one department.
  - Each department has only one value for its budget, and only one associated building.

Functional Dependencies (Cont.)

- An instance of a relation that satisfies all such real-world constraints is called a **legal instance** of the relation;
- A legal instance of a database is one where all the relation instances are legal instances
- Constraints on the set of legal relations.
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes.
- A functional dependency is a generalization of the notion of a *key.*

Functional Dependencies Definition

- Let R be a relation schema

$$\alpha \subseteq R \quad and \quad \beta \subseteq R$$

- The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations r(R), whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \quad \Rightarrow \quad t_1[\beta] = t_2[\beta]$$

- Example: Consider r(A,B) with the following instance of r.

1	4
1	5
3	7

- On this instance, $B \rightarrow A$ hold; $A \rightarrow B$ does **NOT** hold,

Closure of a Set of Functional Dependencies

- Given a set *F* set of functional dependencies, there are certain other functional dependencies that are logically implied by *F*.
  - If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow$ C
  - etc.
- The set of **all** functional dependencies logically implied by *F* is the **closure** of *F*.
- We denote the *closure* of *F* by $\boldsymbol{F^+}$.

Keys and Functional Dependencies

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
  - $K \rightarrow R$, and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

  in_dep (*ID, name, salary, dept_name, building, budget*).

  We expect these functional dependencies to hold:

  dept_name→ *building*

  ID → *building*

  but would not expect the following to hold:

  dept_name → *salary*

Use of Functional Dependencies

- We use functional dependencies to:
  - To test relations to see if they are legal under a given set of functional dependencies.
    - If a relation *r* is legal under a set *F* of functional dependencies, we say that *r* **satisfies** *F*.
  - To specify constraints on the set of legal relations
    - We say that *F* **holds on** *R* if all legal relations on *R* satisfy the set of functional dependencies *F*.
- Note: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances.
  - For example, a specific instance of *instructor* may, by chance, satisfy
    name $\rightarrow$ *ID*.

Trivial Functional Dependencies

- *A* functional dependency is **trivial** if it is satisfied by all instances of a relation
- Example*:*
  - *ID, name* $\rightarrow$ *ID*
  - *name* $\rightarrow$ *name*
- In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

Lossless Decomposition

- We can use functional dependencies to show when certain decomposition are lossless.
- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \prod_{R1}(r) \quad \prod_{R2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless decomposition $\bowtie$ if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies

Example

- $R = (A, B, C)$
  $F = \{A \rightarrow B, B \rightarrow C)$
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless decomposition:
    $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
- $R_1 = (A, B)$, $R_2 = (A, C)$
  - Lossless decomposition:
    $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
- *Note:*
  - $B \rightarrow BC$
    is a shorthand notation for
  - $B \rightarrow \{B, C\}$

Dependency Preservation

- Testing functional dependency constraints each time the database is updated can be costly
- It is useful to design the database in a way that constraints can be tested efficiently.
- If testing a functional dependency can be done by considering just one relation, then the cost of testing this constraint is low
- When decomposing a relation it is possible that it is no longer possible to do the testing without having to perform a Cartesian Produced.
- A decomposition that makes it computationally hard to enforce functional dependency is said to be NOT **dependency preserving**.

Dependency Preservation Example

- Consider a schema:

  dept_advisor(s_ID, i_ID, department_name)

- With function dependencies:

  $i\_ID \rightarrow dept\_name$

  $s\_ID, dept\_name \rightarrow i\_ID$

- In the above design we are forced to repeat the department name once for each time an instructor participates in a *dept_advisor* relationship.
- To fix this, we need to decompose *dept_advisor*
- Any decomposition will not include all the attributes in

  $s\_ID, dept\_name \rightarrow i\_ID$

- Thus, the composition NOT be dependency preserving

First Normal Form (1NF)

- According to the E.F. Codd, a relation will be in 1NF, if each cell of a relation contains only an atomic value.

1NF Example

- Example:

The following Course_Content relation is not in 1NF because the Content attribute contains multiple values.

Course	Content
Programming	Java, c++
Web	HTML, PHP, ASP

1NF Example (Cont..)

- The below relation student is in 1NF:

Course	Content
Programming	Java
Programming	c++
Web	HTML
Web	PHP
Web	ASP

1NF Example - 2 (Cont..)

Simplified version of the COMPANY relational database schema.

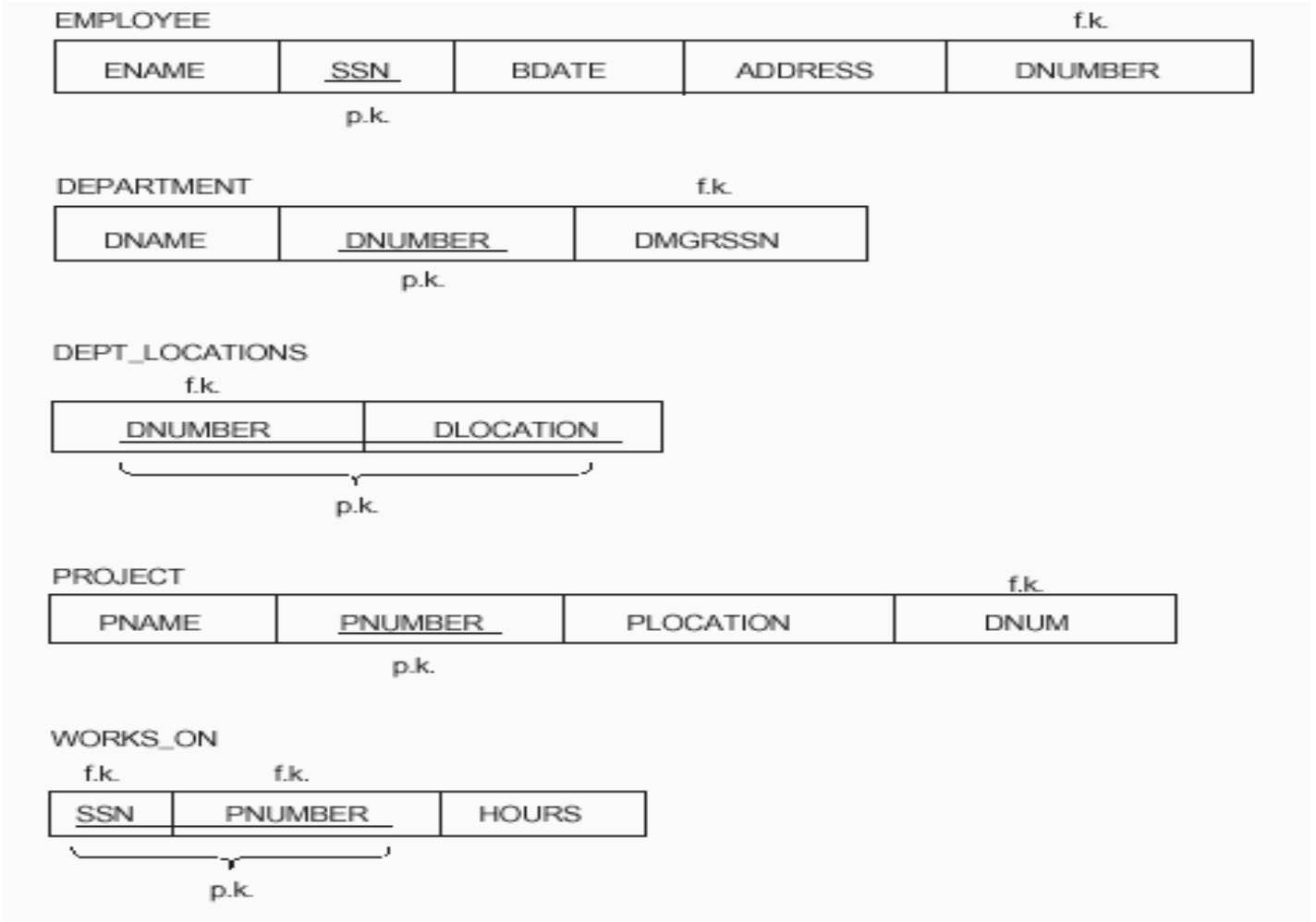
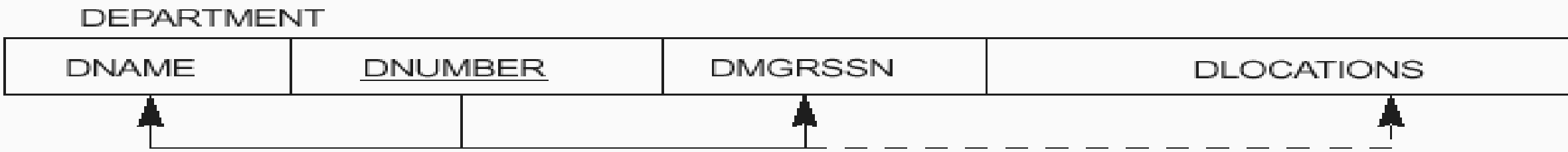


Figure 14.8 Normalization into 1NF. (a) Relational schema that is not in 1NF. (b) Example relation instance. (c) 1NF relation with redundancy.

(a)



(b)

DEPARTMENT

DNAME	DNUMBER	DMGRSSN	DLOCATIONS
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

DNAME	DNUMBER	DMGRSSN	DLOCATION
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 14.9 Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a "nested relation" PROJS. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple.

(a) **EMP_PROJ**

SSN	ENAME	PROJS	
		PNUMBER	HOURS

(b) **EMP_PROJ**

SSN	ENAME	PNUMBER	HOURS
123456789	Smith,John B.	1	32.5
		2	7.5
666884444	Narayan,Ramesh K.	3	40.0
453453453	English,Joyce A.	1	20.0
		2	20.0
333445555	Wong,Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya,Alicia J.	30	30.0
		10	10.0
987987987	Jabbar,Ahmad V.	10	35.0
		30	5.0
987654321	Wallace,Jennifer S.	30	20.0
		20	15.0
888665555	Borg,James E.	20	null

Rules of 1NF

The official qualifications for 1NF are:

1. Each **attribute name** must be unique.
2. Each **attribute value** must be single.
3. Each **row** must be unique.

- Additional:
  - Choose a primary key.
- Reminder:

A primary key is ***unique***, ***not null***, ***unchanged***. A primary key can be either an attribute or combined attributes.

Second Normal Form (2NF)

- According to the E.F. Codd, a relation is in 2NF, if it satisfies the following conditions:
    - The table should be in the First Normal Form.
    - There should be no Partial Dependency.

Prime and Non Prime Attributes

Prime attributes: The attributes which are used to form a candidate key are called prime attributes.

Non-Prime attributes: The attributes which do not form a candidate key are called non-prime attributes.

Roll. No.	First Name of Student	Last Name of Student	Course code
01.	Adam	Gilchrist	A100
02	Adam	Peter	B50
03	John	Gilchrist	C80

- Prime Attribute: Roll No., Course Code
- Non-Prime Attribute: First Name of Student, Last Name of Student

Functional Dependency

- A dependency FD: X → Y means that the values of Y are determined by the values of X. Two tuples sharing the same values of X will necessarily have the same values of Y.
- We illustrate this as:
  - X→ Y *(read as: X determines Y or Y depends on X)*

Functional Dependency

Student ID	Semester	Lecture	TA
1234	6	Numerical Methods	John
1221	4	Numerical Methods	Smith
1234	6	Visual Computing	Bob
1201	2	Numerical Methods	Peter
1201	2	Physics II	Simon

- Whenever two rows in this table feature the same StudentID, they also necessarily have the same Semester values. This basic fact can be expressed by a functional dependency:

$$\text{StudentID} \rightarrow \text{Semester}.$$

Partial Dependency

- If a non-prime attribute can be determined by the part of the candidate key in a relation, it is known as a partial dependency.

Example of partial Dependency: Suppose there is a relation **R** with attributes **A, B,** and **C.**

R(A,B,C)

Where,

{AB} is a candidate key.

{C} is a non-prime attribute.

Then,

{A, B} are the prime attributes.

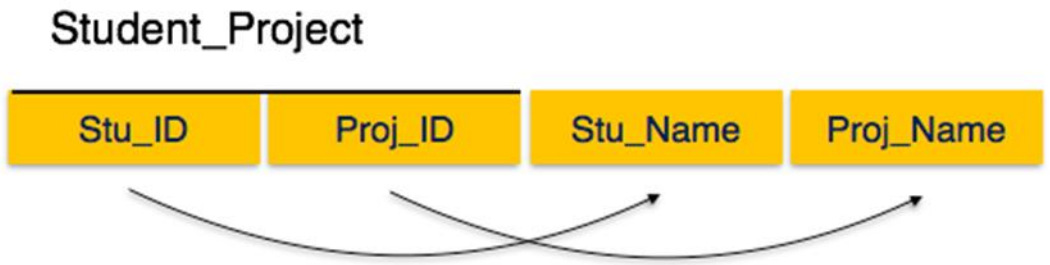
A → C is a partial dependency.

Part of a candidate key

Non- prime attribute

2NF Example

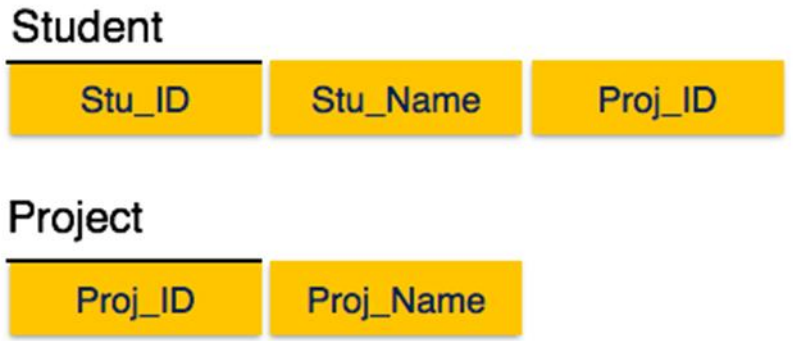
- In Student_Project relation that the prime key attributes are Stu_ID and Proj_ID.
- According to the rule, **non-key attributes**, i.e. **Stu_Name** and **Proj_Name** must be **dependent** upon **both** and **not on any** of the **prime key attribute individually.**
- But we find that Stu_Name can be identified by **Stu_ID** and **Proj_Name** can be identified by **Proj_ID** independently. This is called partial dependency, which is not allowed in Second Normal Form.
- The **non-prime attribute depends** on **one prime attribute not both prime attribute.**



- **Candidate Keys:** {Stu_ID, Proj_ID}
- **Non-prime attribute:** Stu_Name, Proj_Name

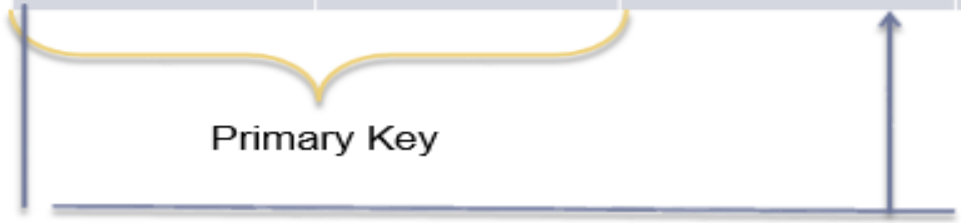
2NF Example (Cont..)

- We broke the relation in two as depicted in the above picture. So there exists no partial dependency.



Example 2NF

CourseID	SemesterID	Num Student	Course Name
IT101	201301	25	Database
IT101	201302	25	Database
IT102	201301	30	Web Prog
IT102	201302	35	Web Prog
IT103	201401	20	Networking

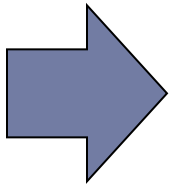


- The Course Name depends on only CourseID, a part of the primary key not the whole primary {CourseID, SemesterID}.It's called partial dependency.
- **Solution:**
- *Remove **CourseID** and **Course Name** together to create a new table.*

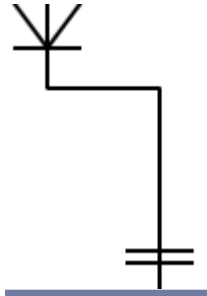
Example 2NF (Cont..)

CourseID	Course Name
IT101	Database
IT101	Database
IT102	Web Prog
IT102	Web Prog
IT103	Networking

Done? Oh no, it is still not in 1NF yet.
Remove the repeating groups too.
Finally, connect the relationship.



CourseID	SemesterID	Num Student
IT101	201301	25
IT101	201302	25
IT102	201301	30
IT102	201302	35
IT103	201401	20



CourseID	Course Name
IT101	Database
IT102	Web Prog
IT103	Networking

Example 2: (c) Decomposing EMP_PROJ into 1NF relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

(c)

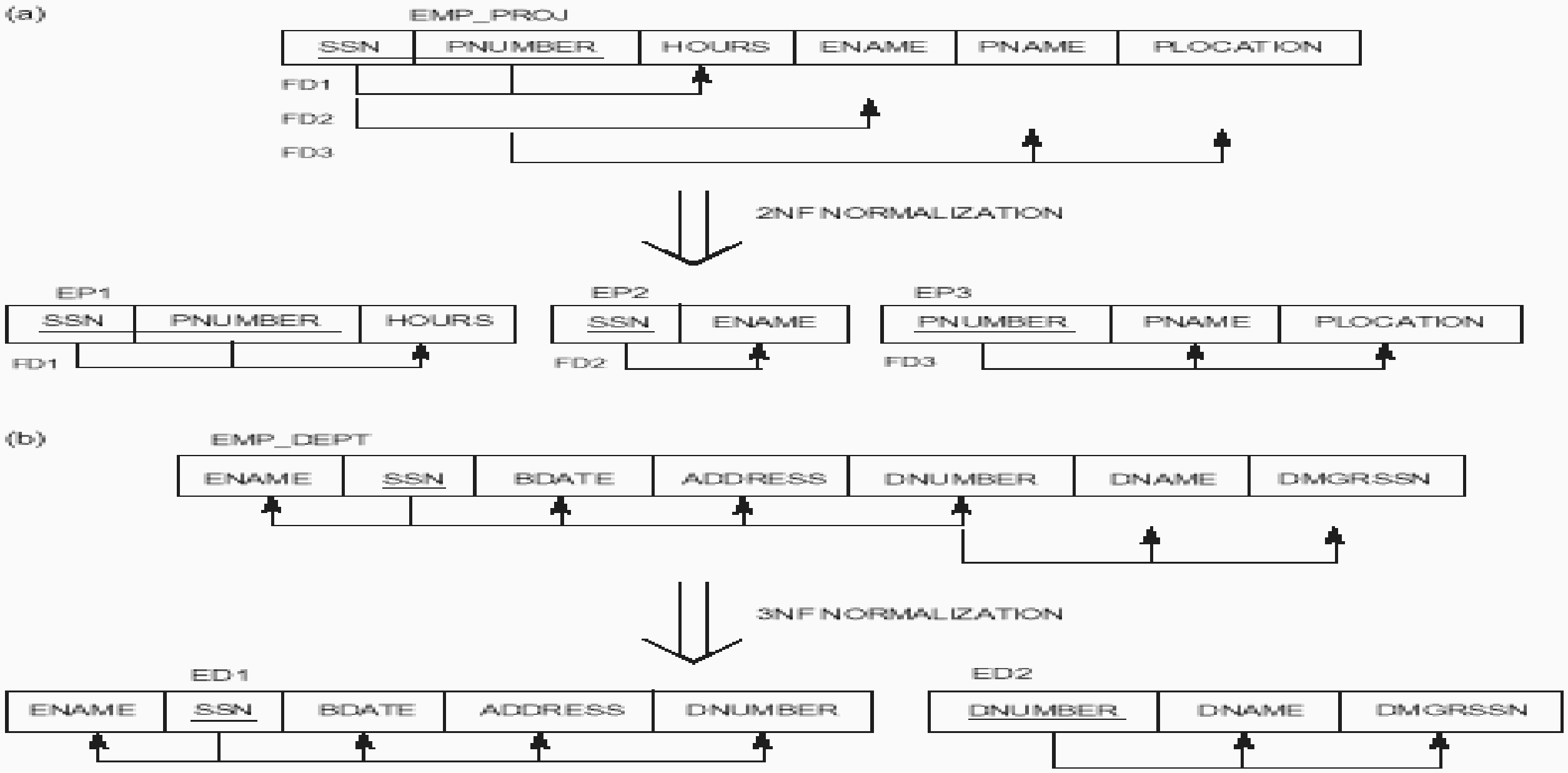
EMP_PROJ1

SSN	ENAME

EMP_PROJ2

SSN	PNUMBER	HOURS

Figure 14.10 The normalization process. (a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.



Third Normal Form (3NF)

- According to the E.F. Codd, a relation is in third normal form (3NF) if it satisfies the following conditions:
  - ✓ It should be in the Second Normal form.
  - ✓ It should not have Transitive Dependency.
  - ✓ All transitive dependencies are removed to place in another table.

Transitive Dependency

- A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies. For e.g.
- X -> Z is a transitive dependency if the following three functional dependencies hold true:

  X->Y

  Y does not ->X

  Y->Z

3NF Example

- We find that in the above Student_detail relation, Stu_ID is the key and only prime key attribute.
- We find that City can be identified by Stu_ID as well as Zip itself.
- Neither Zip is a superkey nor is City a prime attribute. Additionally, **Stu_ID → Zip → City**, so there exists transitive dependency.

Candidate Key: {Prime attribute: Stu_ID

Non-prime attribute: {Stu_Name, City, Zip}

** If city value is null, then zip value is null. Zip -> non-prime attribute depends on another non-prime attribute, city. If this type of relation is exists, then create a separate table for zip,and city by choosing any one attribute as a primary key**.

3NF Example (Cont..)

- To bring this relation into third normal form, we break the relation into two relations as follows −

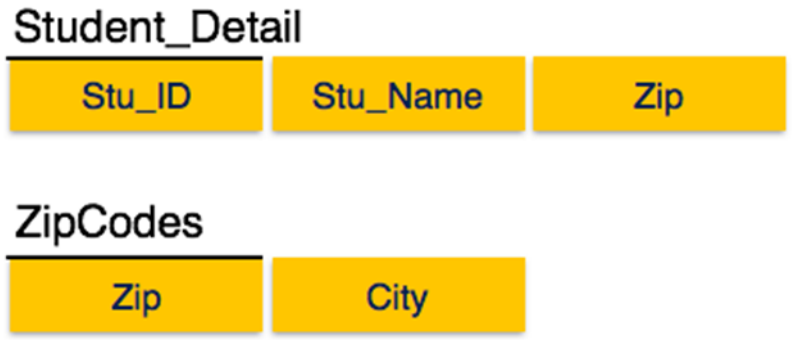
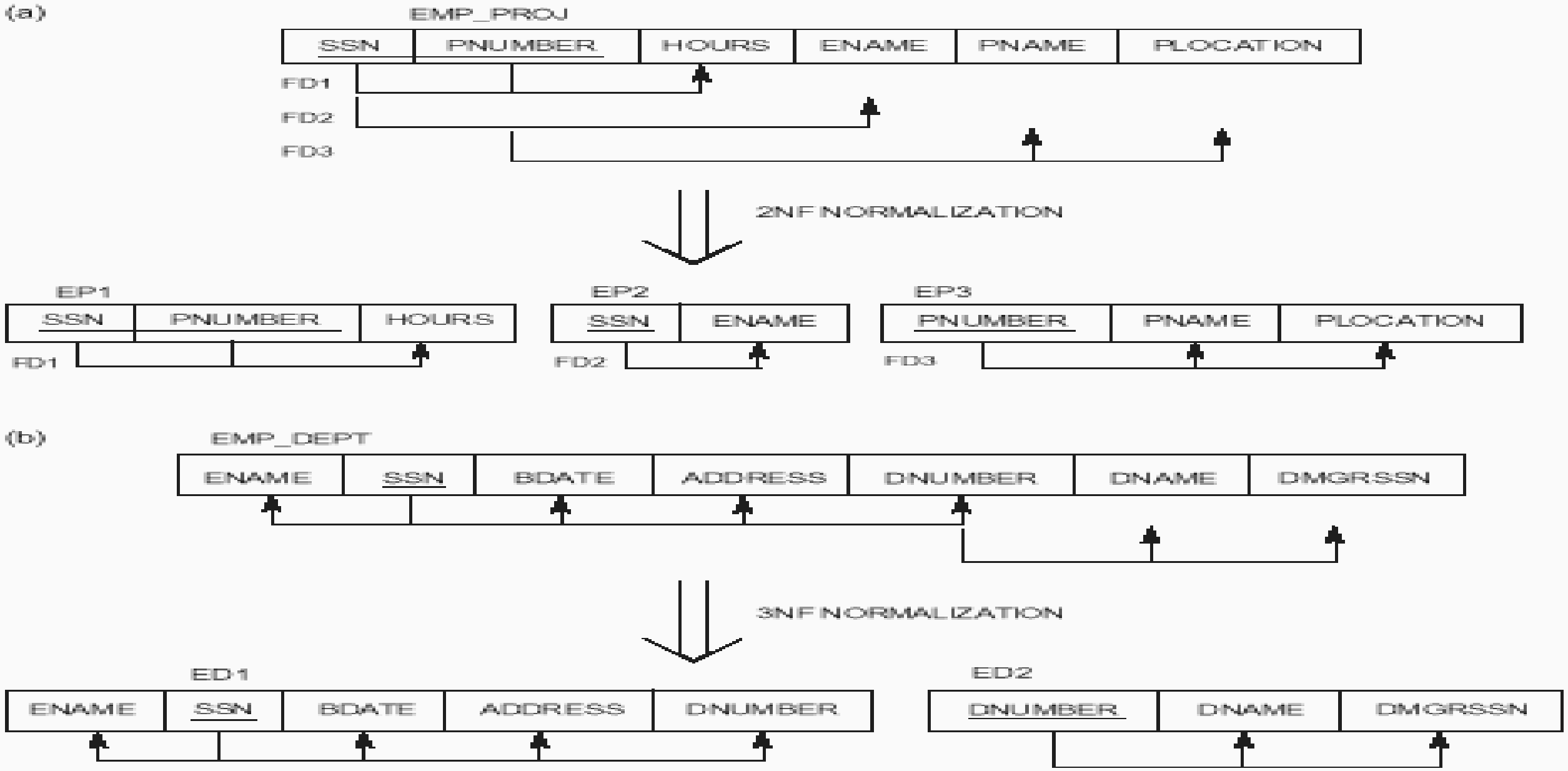


Figure 14.10 The normalization process. (a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.



Example Table

- StudentID is the primary key.

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English Maths Info Tech	$50 $50 $100	B A B+

How can you make it 1NF?

Example 1 (Cont..)

- Create new rows so each cell contains only one value

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

- But now the studentID no longer uniquely identifies each row. You now need to declare *studentID* **and** *subject* **together** to uniquely identify each row. So the new **key** is StudentID *and* Subject.

Is it 2NF?

Example 1 (Cont..)

- **Studentname** and **address** are dependent on studentID (which is part of the key) This is good. But they are **not** dependent on *Subject* (the *other* part of the key)
- **And 2NF requires…**

  All non-key fields are dependent on the ENTIRE key (studentID + subject)

Example 1 (Cont..)

- Make new tables
- Make a new table for each primary key field
- Give each new table its own primary key
- Move columns from the original table to the new table that matches their primary key…

Example (Cont..)

- STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

- RESULTS TABLE (key = StudentID+Subject)

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

But is it 3NF?

Example 1 (Cont..)

- **HouseName** is dependent on both **StudentID + HouseColour**

Or

- **HouseColour** is dependent on both **StudentID + HouseName**

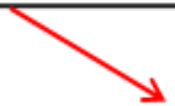
- But either way, non-key fields are dependent on MORE THAN THE PRIMARY KEY (studentID). And 3NF says that non-key fields must depend on **nothing but the key**

Example 1 (Cont..)

StudentTable

StudentID	StudentName	Address	HouseName
19594332X	Mary Watson	10 Charles Street	Bob

Primary key: StudentID



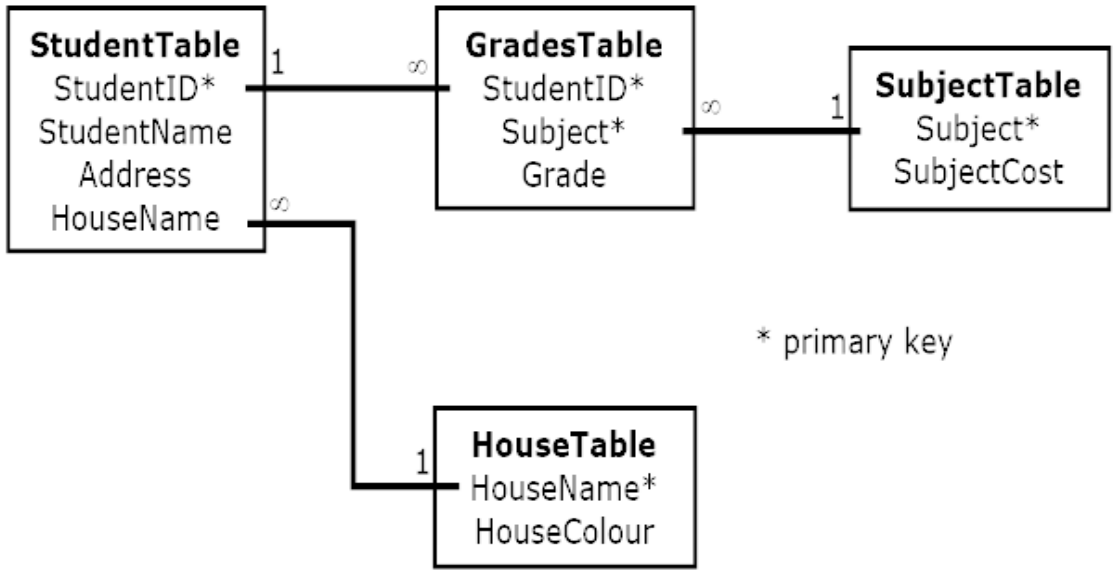
useTable

HouseName	HouseColor
Bob	Red

Primary key: HouseName

Example 1 (Cont..)

- The Final Scheme



Example 2

- We will use the Student_Grade_Report table below, from a School database, as our example to explain the process for 1NF.

Student_Grade_Report (StudentNo, StudentName, Major, CourseNo, CourseName, InstructorNo, InstructorName, InstructorLocation, Grade)

Process for 1NF

- In the Student Grade Report table, the repeating group is the course information. A student can take many courses.
- Remove the repeating group. In this case, it's the course information for each student.
- Identify the PK for your new table.
- The PK must uniquely identify the attribute value (StudentNo and CourseNo).
- After removing all the attributes related to the course and student, you are left with the student course table (StudentCourse).
- The Student table (Student) is now in first normal form with the repeating group removed.
- The two new tables are shown below:

Student (StudentNo, StudentName, Major)

StudentCourse (StudentNo, CourseNo, CourseName, InstructorNo, InstructorName, InstructorLocation, Grade)

Example 2 (Cont..)

Student (StudentNo, StudentName, Major)

StudentCourse (StudentNo, CourseNo, CourseName, InstructorNo, InstructorName, InstructorLocation, Grade)

- To move to 2NF, a table must first be in 1NF.
- The Student table is already in 2NF because it has a single-column PK.
- When examining the Student Course table, we see that not all the attributes are fully dependent on the PK; specifically, all course information. The only attribute that is fully dependent is grade.
- Identify the new table that contains the course information.
- Identify the PK for the new table.
- The three new tables are shown below.

Normal Forms

- A relation R is in *third normal form* if for every FD X → A that holds on R, either

  - X is a superkey of R, or

  - A is a prime attribute of R.

(Alternative Def . - No transitive dependencies – If there is a set of attributes Z that is neither a candidate key nor a subset of any key (primary or candidate) of R , X → Z and Z → Y holds.

SSN → DMGRSSN is transitive as SSN → Dnumber → DMGRSSN (Emp-dept) and dnumber is neither a key nor a subset of key.

- Example. (Fig. 14.10 c)

- A relation R is in *Boyce-Codd normal form* if for every FD X → A that holds on R, X is a superkey of R.

- Example. (Fig. 14.12)

- Increasing Order of restrictiveness: 1NF, 2NF, 3NF, BCNF. For example, if a relation schema R is in BCNF, it is in 3NF.

Figure 14.11 Normalization to 2NF and 3NF. (a) The lots relation schema and its functional dependencies FD1 through FD4. (b) Decomposing lots into the 2NF relations LOTS1 and LOTS2.

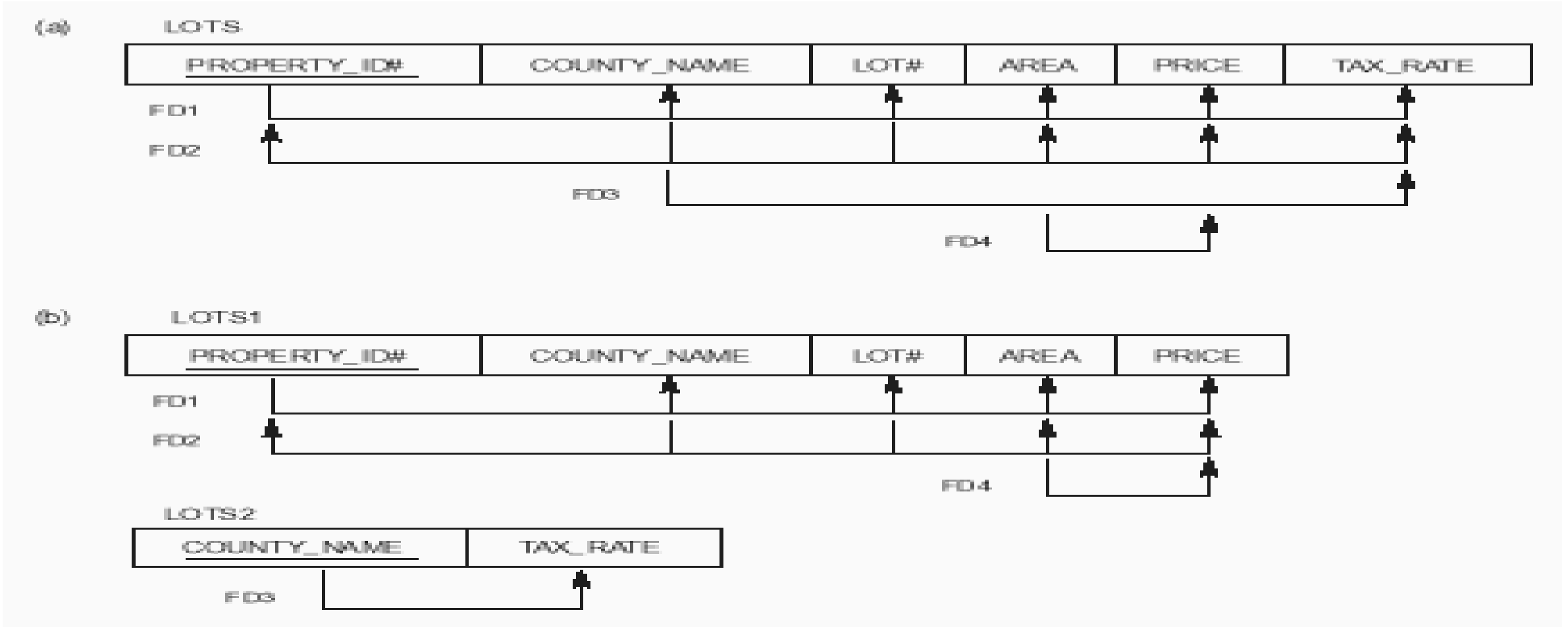


Figure 14.11 (continued)
(c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B.
(d) Summary of normalization of lots.

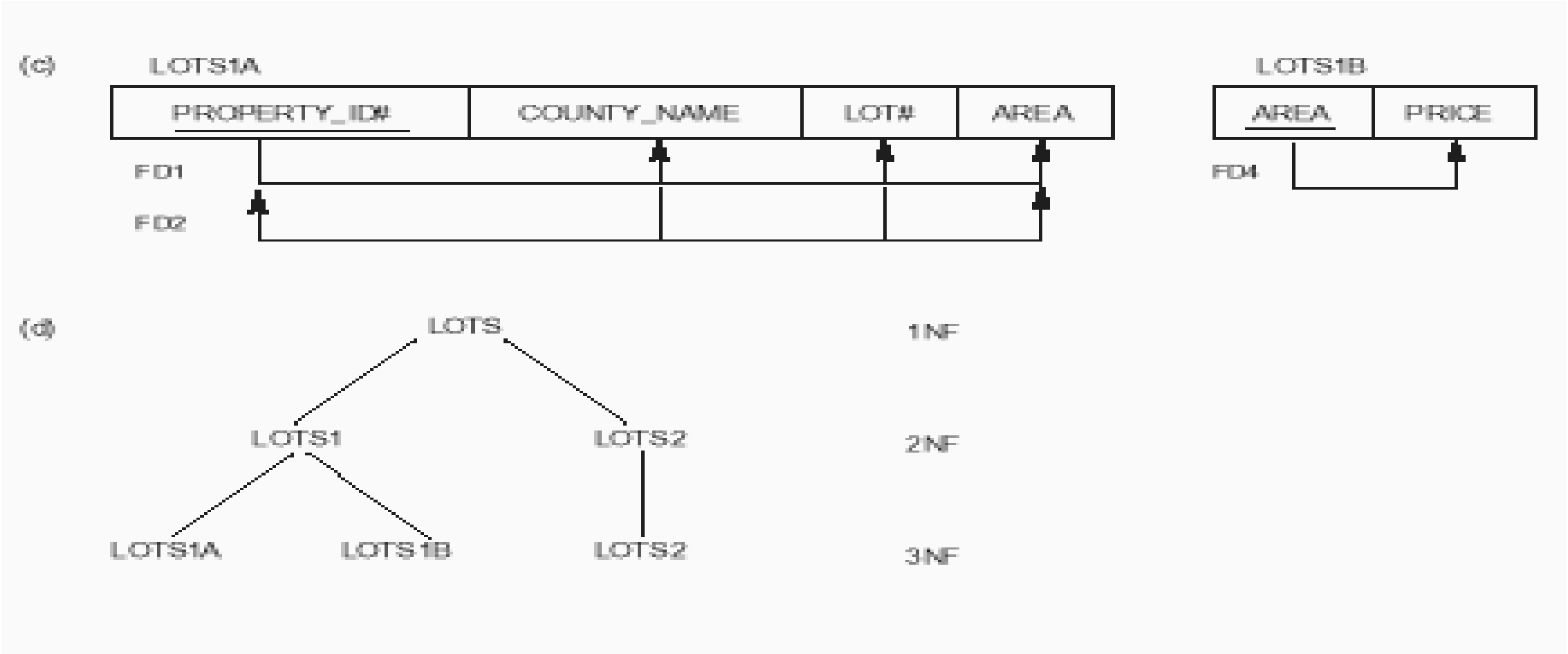


Figure 14.12 Boyce-Codd normal form. (a) BCNF normalization with the dependency of FD2 being "lost" in the decomposition. (b) A relation R in 3NF but not in BCNF.

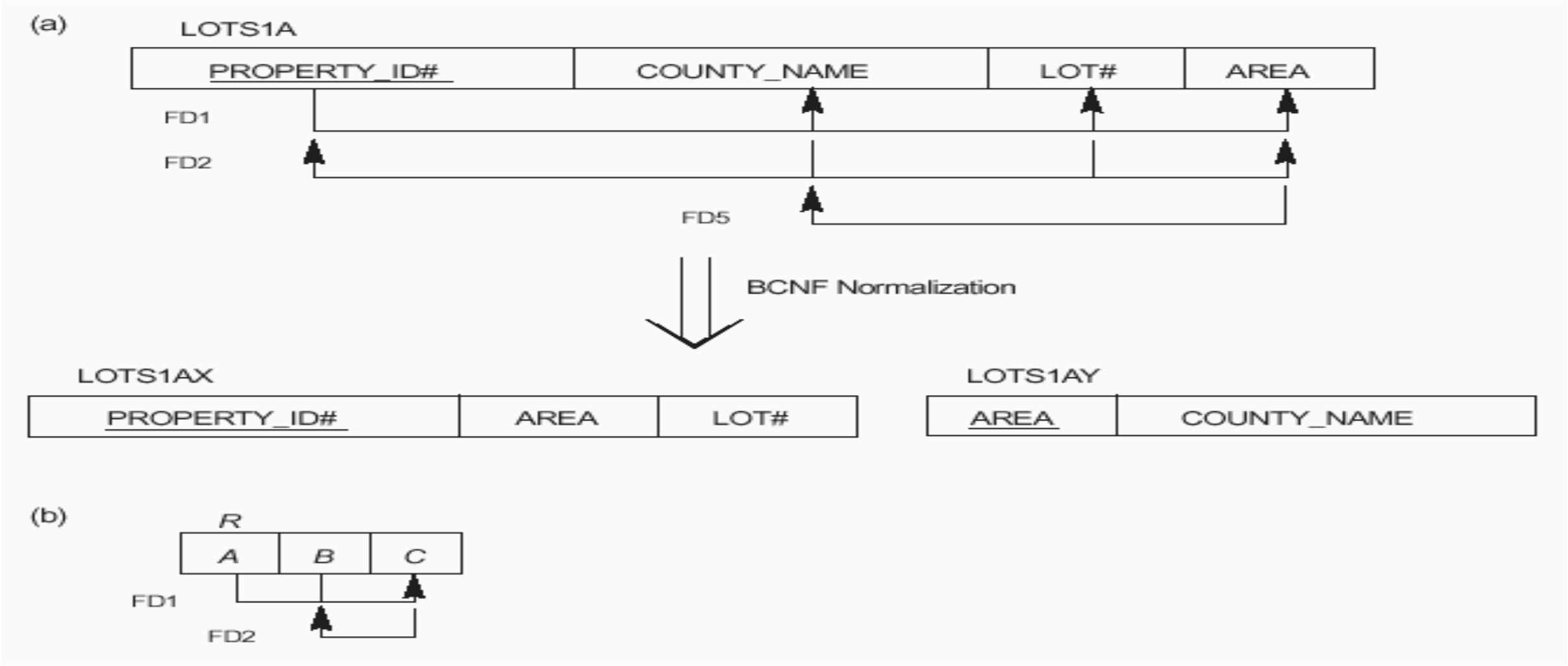


Figure 14.13 A relation TEACH that is in 3NF but not in BCNF.

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

Example 2 (Cont..)

Student (StudentNo, StudentName, Major)

CourseGrade (StudentNo, CourseNo, Grade)

CourseInstructor (CourseNo, CourseName, InstructorNo, InstructorName, InstructorLocation)

Process for 3NF

- Eliminate all dependent attributes in transitive relationship(s) from each of the tables that have a transitive relationship.
- Create new table(s) with removed dependency.
- Check new table(s) as well as table(s) modified to make sure that each table has a determinant and that no table contains inappropriate dependencies.
- See the four new tables below.

Process for 3NF

Student (StudentNo, StudentName, Major)

CourseGrade (StudentNo, CourseNo, Grade)

Course (CourseNo, CourseName, InstructorNo)

Instructor (InstructorNo, InstructorName, InstructorLocation)

Process for 3NF

- At this stage, there should be no anomalies in third normal form.

Student (StudentNo, StudentName, Major)

StudentCourse (StudentNo, CourseNo, CourseName, InstructorNo, InstructorName, InstructorLocation, Grade)